National University of Computer and Emerging Sciences

FAST School of Computing **Fall-2023** **Islamabad Campus**

Question 1 [15+10 Marks]

(a) Write a function that will calculate the number of 1s and 0s in the binary of a given number using bitwise operators.

(b) Write a C++ program that calculates the value of the following mathematical equation for a given value of 'x'. Your program should take the value of 'x' as input and display the result.

$$f(x) = \frac{(x^3+5x^2-4x+3)}{(2x^4+7x^3-3x^2+6)} - \sqrt[3]{\frac{(x^2-3x+2)}{(4x^3-5x^2+8)}}$$

Question 2 [25 Marks]

Write a C++ program for a vending machine that sells various products, such as snacks and drinks. The program should allow users to select a product, specify their payment method (either cash or card), and then calculate the change if the user pays with cash or process the payment if a card is used. To accomplish this, you should use a nested switch statement.

Requirements:

1. The program should display a menu of available products with their corresponding prices. For example:

```
Vending Machine Menu:
1. Snack A - $1.50
2. Snack B - $2.00
3. Drink A - $1.00
4. Drink B - $1.25
```

2. Prompt the user to select a product by entering the corresponding product number (1, 2, 3, or 4). Handle invalid product selections appropriately.
3. Ask user if he wants to select more products until he enters -1
4. If user has entered -1, the program should prompt the user to choose a payment method: "Cash" or "Card."

5. If the user chooses "Cash," they should be prompted to enter the amount in dollars. The program should calculate and display the change to the user. If the entered amount is less than total bill display the message "Insufficient balance"

6. If the user chooses "Card," simulate a card payment processing by displaying a message such as "Card payment successful."

Question 3 [25 Marks]

Case Study: Text-Based Maze Escape Game

Introduction:
In this case study, you will create a simple text-based maze escape game using C++. The game allows a player to navigate through a maze and reach the exit while avoiding obstacles.

Objective:
The objective of this case study is to create a basic text-based game where the player moves through a maze represented by a grid and reaches the exit.

Implementation:
1. Defining the Maze:
We start by defining the maze grid as a string, where each character represents a cell in the maze. The maze includes walls ('#'), an exit ('E'), and an initial position for the player ('P'). This is how the maze is represented:

```
########
#P#  #E#
# # # #
#      #
########
```

You need to hard code this maze as it is.

'#' represents walls, which block movement.
'P' represents the player's initial position.
'E' represents the exit, which is the goal of the game.

2. Game Loop:
You need to implement a game loop that continues until the player reaches the exit (**'E'**). The loop performs the following steps:
Display the current state of the maze.
Prompt the player for a move (**W/A/S/D** for up/left/down/right).
Update the player's position based on the move.
Check for collisions with walls or the exit.
Repeat these steps until the player reaches the exit.

3. Player's Movement:

The player can move in four directions:
'W': Move up (if no wall blocks the way).
'A': Move left (if no wall blocks the way).
'S': Move down (if no wall blocks the way).
'D': Move right (if no wall blocks the way).

4. Win Condition:
When the player's position reaches the exit ('E'), the game displays a win message, and the loop ends.

5. Input and Output:
The game uses **cin** to get input from the player and **cout** to display the game's state and messages on the console.

Question 4 [25 Marks]

Write a C++ program to simulate a Ludo, Snake, and Ladder game for two players. The game will have a 10x10 game board with snakes and ladders. Define the positions of snakes and ladders on the board on random basis. Create a function to roll a six-sided dice to generate a random result and return the value (e.g., **int RollDice()**).

Implement a function to move a player's game piece based on the dice roll and the current position of the player (e.g., **int MovePlayer(int player, int currentPosition, int diceRoll)**). This function should consider snakes and ladders, and it should return the new position of the player. Display the game board with players' positions and the positions of snakes and ladders (e.g., **void DisplayBoard(int player1Pos, int player2Pos)**). Initial positions of both the players would be 1.

The next position will be determined by adding the dice value to the initial position. Simulate the game for two players. Players take turns rolling the dice and moving their game pieces. The game continues until one player reaches or surpasses the final square (100). Declare the winner of the game at the end.
Function Prototypes:

1. `int RollDice();`
2. `int MovePlayer(int player, int currentPosition, int diceRoll);`
3. `int CheckForLadder(int square);`
4. `int CheckForSnake(int square);`
5. `void DisplayBoard(int player1Pos, int player2Pos);`
6. `void PlayLudoSnakeLadderGame();`

Sample Output:

```
Welcome to the Ludo, Snake, and Ladder Game!

Game Board:
[100] [99] [98] [97] [96] [95] [94] [93] [92] [91]
[81]  [82] [83] [84] [85] [86] [87] [88] [89] [90]
[80]  [79] [78] [77] [76] [75] [74] [73] [72] [71]
[61]  [62] [63] [64] [65] [66] [67] [68] [69] [70]
[60]  [59] [58] [57] [56] [55] [54] [53] [52] [51]
[41]  [42] [43] [44] [45] [46] [47] [48] [49] [50]
[40]  [39] [38] [37] [36] [35] [34] [33] [32] [31]
[21]  [22] [23] [24] [25] [26] [27] [28] [29] [30]
[20]  [19] [18] [17] [16] [15] [14] [13] [12] [11]
[1]  [2] [3] [4] [5] [6] [7] [8] [9] [10]

Ladders:
3 -> 22 (Climb up)
8 -> 39 (Climb up)
18 -> 72 (Climb up)
25 -> 80 (Climb up)

Snakes:
24 -> 4 (Slide down)
34 -> 15 (Slide down)
44 -> 36 (Slide down)
60 -> 23 (Slide down)
```

```
Player 1 (P1) is at square 1.
Player 2 (P2) is at square 1.

Player 1's turn (press enter to roll the dice)
Player 1 (P1) rolled a 5. Moving to square 6.

Game Board:
[100] [99] [98] [97] [96] [95] [94] [93] [92] [91]
[81]  [82] [83] [84] [85] [86] [87] [88] [89] [90]
[80]  [79] [78] [77] [76] [75] [74] [73] [72] [71]
[61]  [62] [63] [64] [65] [66] [67] [68] [69] [70]
[60]  [59] [58] [57] [56] [55] [54] [53] [52] [51]
[41]  [42] [43] [44] [45] [46] [47] [48] [49] [50]
[40]  [39] [38] [37] [36] [35] [34] [33] [32] [31]
[21]  [22] [23] [24] [25] [26] [27] [28] [29] [30]
[20]  [19] [18] [17] [16] [15] [14] [13] [12] [11]
[1]  [2] [3] [4] [5] [P1] [7] [8] [9] [10]

...
```